

AlphaAI Fund Platform

Complete Technical Documentation

For External Review & System Rebuilding

Document Type: Complete System Export
Generated: March 17, 2026 at 09:17
Version: 2.0 (Comprehensive)
Platform Status: Testnet Ready
Copyright: © 2026 Martin Maughan. All rights reserved.

Table of Contents

1. Executive Summary & Project Overview
2. System Architecture
 - 2.1 Technology Stack
 - 2.2 Directory Structure
 - 2.3 Data Flow
3. Backend Implementation (server.py)
 - 3.1 Data Models (Pydantic)
 - 3.2 Simulation Engine
 - 3.3 API Endpoints - Full Reference
 - 3.4 AI Agent Logic
 - 3.5 Risk Management Engine
 - 3.6 Research Engine
 - 3.7 Smart Contract Integration
 - 3.8 Event-Driven Agent System
4. Frontend Implementation (App.js)
 - 4.1 React Components
 - 4.2 Page Structure
 - 4.3 Wallet Integration (MetaMask)
 - 4.4 State Management
5. Smart Contract (Solidity)
 - 5.1 Contract Source Code
 - 5.2 Functions & Events
 - 5.3 Deployment Guide
6. Trading Strategies & Signal Logic
 - 6.1 Strategy Types
 - 6.2 Trading Rules
 - 6.3 Backtesting Logic
7. AI Integration & Prompts
8. Database Schema
9. Configuration Files
10. End-to-End System Workflow

1. Executive Summary & Project Overview

1.1 What is AlphaAI?

AlphaAI is a decentralized AI-powered hedge fund platform that enables investors to deposit capital into a vault managed by autonomous AI trading agents. The platform features a marketplace where external developers can launch their own trading agents, comprehensive simulation capabilities with accelerated backtesting, real-time market data integration, and smart contract integration for on-chain fund management.

1.2 Core Capabilities

- **Multi-Agent Trading System:** 4 specialized AI agents (Arbitrage, Momentum, Funding Rate, Research Lab) trade autonomously
- **AI Research Engine:** Generates, backtests, and deploys new strategies with walk-forward validation
- **500x Accelerated Simulation:** Run 6 months of trading in minutes using historical data
- **Smart Contract Integration:** AlphaAIManager.sol for on-chain deposits, withdrawals, and strategy allocation
- **MetaMask Wallet:** Real Web3 wallet connectivity on Sepolia testnet
- **Live Price Feed:** Real-time crypto prices from Kraken API
- **Risk Management:** Automated drawdown limits, stop-losses, and position sizing
- **Event-Driven Agents:** Agents that react to smart contract events automatically

1.3 Key Platform Statistics

Metric	Value
Backend Code	~3,400 lines (server.py)
Frontend Code	~3,100 lines (App.js)
API Endpoints	60+ REST endpoints
Data Models	25+ Pydantic models
React Components	15+ components
Trading Strategies	5 types (momentum, arbitrage, yield, mean_reversion, funding)
Supported Cryptocurrencies	8 (BTC, ETH, SOL, AVAX, MATIC, LINK, UNI, AAVE)

2. System Architecture

2.1 Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Shadcn/UI	User interface components
Styling	TailwindCSS + Framer Motion	Design system and animations
Charts	Recharts	Data visualization
Web3 (Frontend)	ethers.js v5	MetaMask wallet integration
Backend	FastAPI (Python 3.11+)	REST API server
Database	MongoDB (Motor async)	Data persistence
AI/LLM	OpenAI GPT-5.2	Market analysis via Emergent Key
Price Data	Kraken API	Real-time crypto prices
Web3 (Backend)	web3.py	Smart contract interaction
PDF Reports	ReportLab	Investor report generation
Blockchain	Ethereum (Sepolia)	Smart contract deployment

2.2 Directory Structure

```
/app/
├── backend/
│   ├── server.py # Main FastAPI application (3,400 lines)
│   ├── requirements.txt # Python dependencies
│   ├── .env # Environment variables (MONGO_URL, EMERGENT_LLM_KEY)
│   └── contracts/
│       └── AlphaAIManager.sol # Solidity smart contract
│   └── data/
│       ├── btc_usd.csv # Historical BTC price data
│       ├── eth_usd.csv # Historical ETH price data
│       └── reports/
│           └── investor_reports/ # Generated PDF/JSON reports
│   └── web3/
│       └── contract_abi.py # Smart contract ABI
├── frontend/
│   └── src/
│       ├── App.js # Main React application (3,100 lines)
│       ├── App.css # Global styles
│       ├── index.css # TailwindCSS imports
│       └── components/ui/ # Shadcn/UI components
│   ├── package.json # NPM dependencies
│   ├── .env # REACT_APP_BACKEND_URL
│   └── memory/
└── PRD.md # Product requirements document
```

2.3 Data Flow Architecture

User Flow: User connects MetaMask → Frontend calls `/api/investors/register` → Backend creates investor record in MongoDB → User can deposit/withdraw via simulated or on-chain transactions.

Trading Flow: Simulation Engine runs → Calls strategy cycle → Each agent generates signals → Execution layer simulates trades → Risk engine checks rules → Capital allocation engine rebalances → Results logged to database.

Event Flow: Smart contract emits event (e.g., `InvestorDeposited`) → Event-driven agents detect event → Execute automated actions (update balances, allocate to strategies) → Update dashboards.

3. Backend Implementation (server.py)

The backend is a monolithic FastAPI application contained in a single file (server.py). This section documents all data models, the simulation engine, API endpoints, and core logic.

3.1 Data Models (Pydantic)

Total Models Defined: 34

Model Name	Key Fields	Purpose
Investor	wallet_address, balance, shares, paper_balance	Track investor accounts
TradingAgent	name, type, strategy, performance_7d, capital_allocation	AI agent definitions
Trade	agent_id, symbol, side, amount, price, pnl, slippage	Trade records
Strategy	name, type, parameters, status, sharpe_ratio, total_return	Trading strategies
RiskConfig	max_drawdown, max_position_size, max_daily_loss, stop_loss	Risk parameters
SimulationConfig	is_running, mode, initial_capital, time_acceleration	Simulation settings
RiskAlert	type, severity, message, auto_action_taken	Risk alerts
EventAgent	name, type, events_to_monitor, is_active	Event-driven agents
ContractEvent	event_name, block_number, tx_hash, args	Smart contract events

3.1.1 Complete Model Definitions

```
class Investor(BaseModel):
    model_config = ConfigDict(extra="ignore")
    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    wallet_address: str
    balance: float = 0.0
    shares: float = 0.0
    total_deposited: float = 0.0
    total_withdrawn: float = 0.0
    paper_balance: float = 10000.0
    paper_pnl: float = 0.0
    is_paper_trading: bool = False
    created_at: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))

class TradingAgent(BaseModel):
    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    name: str
    type: str # data, analysis, strategy, execution, risk
    status: str = "active"
    description: str
    strategy: str
    performance_7d: float = 0.0
    performance_30d: float = 0.0
    total_trades: int = 0
    win_rate: float = 0.0
    aum: float = 0.0
    capital_allocation: float = 0.0

class Strategy(BaseModel):
    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    name: str
    type: str # momentum, arbitrage, yield, mean_reversion, funding
    description: str
    parameters: Dict[str, Any] = {}
    status: str = "generated" # generated, backtested, sandbox, live, stopped
    backtest_results: Optional[Dict[str, Any]] = None
    sharpe_ratio: float = 0.0
    total_return: float = 0.0
    max_drawdown: float = 0.0
    win_rate: float = 0.0
    capital_allocated: float = 0.0
    is_active: bool = False

class SimulationConfig(BaseModel):
    is_running: bool = False
    mode: str = "paper" # paper, testnet, live
    initial_capital: float = 10000.0
    current_capital: float = 10000.0
```

```
time_acceleration: int = 1 # 1x = real-time, 500x = accelerated
sim_start_date: str = "2025-01-01"
sim_end_date: str = "2025-12-31"
stress_test_active: bool = False

class RiskConfig(BaseModel):
    max_drawdown: float = 5.0 # Auto-stop at 5% drawdown
    max_position_size: float = 10.0 # Max 10% per position
    max_daily_loss: float = 2.0 # Reduce positions at 2% daily loss
    stop_loss: float = 2.0 # Per-agent stop loss
    auto_shutdown_enabled: bool = True
```

3.2 Simulation Engine

The `SimulationEngine` class is the core of the trading simulation. It coordinates all agents, manages capital allocation, executes trades, and enforces risk rules.

```
class SimulationEngine:
    '''Central simulation engine coordinating all agents and modules'''

    def __init__(self):
        self.is_running = False
        self.config = None
        self.current_drawdown = 0.0
        self.daily_pnl = 0.0
        self.positions = {}

    async def initialize(self):
        '''Initialize or load simulation config from database'''
        config = await db.simulation_config.find_one({}, {"_id": 0})
        if not config:
            self.config = SimulationConfig()
            await db.simulation_config.insert_one(self.config.model_dump())
        else:
            self.config = SimulationConfig(**config)
        return self.config

    async def check_risk_rules(self) -> Dict[str, Any]:
        '''Risk Engine: Check if any risk rules are triggered'''
        violations = []
        actions_taken = []

        # Check drawdown against limit
        if self.current_drawdown >= risk_config['max_drawdown']:
            violations.append({"rule": "max_drawdown", "value": self.current_drawdown})
            if risk_config['auto_shutdown_enabled']:
                actions_taken.append("AUTO_STOP_ALL_TRADING")

        # Check daily loss
        if abs(self.daily_pnl) >= risk_config['max_daily_loss'] and self.daily_pnl < 0:
            violations.append({"rule": "max_daily_loss", "value": self.daily_pnl})
            actions_taken.append("REDUCE_POSITION_SIZES")

        return {"violations": violations, "actions_taken": actions_taken}

    async def dynamic_capital_allocation(self) -> List[Dict]:
        '''Capital Allocation Engine: Reallocate based on performance'''
        strategies = await db.strategies.find({"status": {"$in": ["live", "sandbox"]}}).to_list(100)

        # Calculate performance scores
        for s in strategies:
            s['performance_score'] = (s['sharpe_ratio'] * 0.4 +
                                      s['total_return'] * 0.3 +
                                      (100 - s['max_drawdown'] * 0.3))

        # Sort by performance and allocate proportionally
        strategies = sorted(strategies, key=lambda x: x['performance_score'], reverse=True)
        total_score = sum(max(s['performance_score'], 1) for s in strategies)

        allocations = []
        for s in strategies:
            alloc_percent = (s['performance_score'] / total_score) * 100
            alloc_capital = self.config.current_capital * (alloc_percent / 100)
            allocations.append({
                "strategy_id": s['id'],
                "allocation_percent": round(alloc_percent, 2),
                "allocated_capital": round(alloc_capital, 2)
            })

        return allocations

    async def execute_simulation_trade(self, strategy_id, symbol, side, amount) -> Trade:
        '''Execution Layer: Execute a simulated trade'''
        prices = {"BTC/USDT": 45000, "ETH/USDT": 2500, "SOL/USDT": 100}
        base_price = prices.get(symbol, 100)

        # Calculate slippage and execution
        slippage = round(random.uniform(0.01, 0.3), 3)
        execution_price = base_price * (1 + slippage/100) if side == "buy" else base_price * (1 - slippage/100)
        gas_fee = round(random.uniform(1, 10), 2)

        # Calculate P&L
        price_movement = random.uniform(-0.02, 0.03)
        pnl = round(amount * execution_price * price_movement, 2)

        trade = Trade(
            agent_id="ExecutionAgent",
            strategy_id=strategy_id,
            symbol=symbol,
            side=side,
            amount=amount,
```

```
price=base_price,
execution_price=round(execution_price, 2),
slippage=slippage,
gas_fee=gas_fee,
pnl=pnl,
is_simulation=True
)

await db.trades.insert_one(trade.model_dump())
await self.check_risk_rules() # Check risk after every trade

return trade
```

3.3 API Endpoints - Full Reference

Total API Endpoints: **85**

Simulation Endpoints

Method	Path	Description
GET	/simulation/config	Get simulation configuration
POST	/simulation/start	Start simulation in paper mode
POST	/simulation/stop	Stop active simulation
POST	/simulation/run-cycle	Run one trading cycle
GET	/simulation/logs	Get simulation event logs
GET	/simulation/agent-interactions	API endpoint
GET	/simulation/stats	Get comprehensive statistics
POST	/simulation/configure	Get simulation configuration
POST	/simulation/run-accelerated	Run 500x accelerated simulation
POST	/simulation/stress-test	Run stress test scenario
GET	/simulation/stress-scenarios	API endpoint
GET	/simulation/agent-performance	API endpoint
POST	/simulation/export	Export simulation results
POST	/simulation/load-historical-data	API endpoint
POST	/simulation/switch-mode	API endpoint

Research Endpoints

Method	Path	Description
GET	/research/data-sources	Get available data sources
POST	/research/run-simulation	Run research backtest with CSV data
POST	/research/walk-forward-test	Run walk-forward validation
POST	/research/generate-investor-report	Generate investor PDF report
GET	/research/reports/{report_id}	API endpoint
GET	/research/reports	API endpoint
GET	/research/metrics	API endpoint

Strategy Lab Endpoints

Method	Path	Description
POST	/strategies/add-batch	API endpoint
POST	/lab/auto-deploy-top	Auto-deploy top strategies
GET	/lab/strategies	Get all strategies
POST	/lab/strategies/generate	Get all strategies
POST	/lab/strategies/{strategy_id}/backtest	Get all strategies
POST	/lab/strategies/{strategy_id}/sandbox	Get all strategies
POST	/lab/strategies/{strategy_id}/deploy	Get all strategies
POST	/lab/strategies/{strategy_id}/stop	Get all strategies

GET	/lab/rankings	Get strategy rankings
GET	/analytics/strategies	API endpoint
GET	/contract/strategies	API endpoint

Investors Endpoints

Method	Path	Description
POST	/investors/register	Register new investor
GET	/investors/{wallet_address}	API endpoint
POST	/investors/deposit	Deposit funds
POST	/investors/withdraw	Withdraw funds
POST	/paper/trade	API endpoint
GET	/paper/portfolio/{wallet_address}	API endpoint
POST	/paper/reset/{wallet_address}	API endpoint
POST	/investors/toggle-paper-trading/{wallet_address}	API endpoint
POST	/research/generate-investor-report	Generate investor PDF report
GET	/dashboards/investor-balances	API endpoint

Trading Agents Endpoints

Method	Path	Description
GET	/simulation/agent-interactions	API endpoint
GET	/simulation/agent-performance	API endpoint
POST	/agents/add	Get all trading agents
GET	/agents	Get all trading agents
GET	/marketplace/agents	Get all trading agents
POST	/marketplace/agents	Get all trading agents
GET	/agents/event-agents	Get all trading agents
POST	/agents/event-agents/toggle/{agent_id}	Get all trading agents

Risk Management Endpoints

Method	Path	Description
GET	/risk/config	Get/update risk configuration
PUT	/risk/config	Get/update risk configuration
GET	/risk/alerts	Get active risk alerts
POST	/risk/alerts/{alert_id}/resolve	Get active risk alerts
GET	/risk/portfolio-status	Get portfolio risk status

Market Data Endpoints

Method	Path	Description
GET	/market/top-coins	Get top coins by market cap
GET	/market/chart/{symbol}	API endpoint

GET	/market/live-prices	Get live prices from Kraken
GET	/marketplace/agents	Get all trading agents
POST	/marketplace/agents	Get all trading agents

Smart Contract Endpoints

Method	Path	Description
GET	/contract/info	Get contract deployment info
GET	/contract/source	Get Solidity source code
POST	/contract/register	Register deployed contract
GET	/contract/balance/{wallet_address}	API endpoint
GET	/contract/strategies	API endpoint
POST	/contract/prepare-deposit	API endpoint
POST	/contract/prepare-withdraw	API endpoint
GET	/contract/events	API endpoint
GET	/contract/deployment-guide	API endpoint
GET	/contract/abi	Get contract ABI

Events Endpoints

Method	Path	Description
GET	/contract/events	API endpoint
GET	/agents/event-agents	Get all trading agents
POST	/agents/event-agents/toggle/{agent_id}	Get all trading agents
POST	/events/simulate	Simulate contract event
GET	/events/recent	Get recent contract events

Reports Endpoints

Method	Path	Description
GET	/reports/daily	API endpoint
GET	/reports/weekly	API endpoint
GET	/reports/history	API endpoint
POST	/research/generate-investor-report	Generate investor PDF report
GET	/research/reports/{report_id}	API endpoint
GET	/research/reports	API endpoint

Fund Stats Endpoints

Method	Path	Description
GET	/fund/stats	Get fund statistics
GET	/fund/allocation	Get portfolio allocation
GET	/fund/performance-history	API endpoint

Other Endpoints

Method	Path	Description
GET	/	API endpoint
POST	/status	API endpoint
GET	/status	API endpoint
GET	/trades	API endpoint
GET	/capital/allocations	API endpoint
POST	/capital/rebalance	API endpoint
GET	/execution/stats	API endpoint
POST	/execution/simulate	API endpoint
POST	/ai/analyze	Run AI market analysis
GET	/analytics/overview	API endpoint
GET	/dashboards/strategy-allocation	API endpoint
GET	/integration/config	API endpoint

4. Frontend Implementation (App.js)

The frontend is a single-page React application with comprehensive UI for investors, administrators, and traders. It uses Shadcn/UI components and ethers.js for Web3 integration.

4.1 React Components

Component	Purpose	Key Features
WalletProvider	Context for wallet state	MetaMask connection, balance tracking
Navigation	Top navigation bar	Responsive nav, wallet dropdown
LivePriceTicker	Real-time price display	Kraken API, 30s refresh
LandingPage	Homepage	Hero section, fund stats, features
DashboardPage	Investor dashboard	Portfolio, charts, paper trading
StrategyLabPage	Strategy management	Generate, backtest, deploy
AgentsPage	AI agents view	Agent cards, AI analysis
MarketplacePage	Agent marketplace	Browse, submit agents
AnalyticsPage	Performance analytics	Charts, metrics, rankings
AdminPage	Admin dashboard	Risk config, alerts, monitoring
SimulationPage	Simulation control	Start/stop, stress tests
ResearchPage	Research engine	Backtesting, walk-forward
EventAgentsPage	Event-driven agents	Monitor, toggle agents

4.2 Page Structure & Routes

Route	Component	Description
/	LandingPage	Public homepage with fund stats
/dashboard	DashboardPage	Investor portfolio and paper trading
/simulation	SimulationPage	Simulation control panel
/research	ResearchPage	AI Research Engine interface
/agents	AgentsPage	View AI trading agents
/events	EventAgentsPage	Event-driven agent monitoring
/lab	StrategyLabPage	Strategy generation and testing
/marketplace	MarketplacePage	Agent marketplace
/admin	AdminPage	Admin dashboard with risk config

4.3 MetaMask Wallet Integration

```
import { ethers } from "ethers";

const WalletProvider = ({ children }) => {
  const [wallet, setWallet] = useState(null);
  const [provider, setProvider] = useState(null);
  const [signer, setSigner] = useState(null);
  const [chainId, setChainId] = useState(null);
  const [ethBalance, setEthBalance] = useState(null);

  const connectWallet = async () => {
```

```

if (typeof window.ethereum !== 'undefined') {
  // Request accounts from MetaMask
  const accounts = await window.ethereum.request({ method: 'eth_requestAccounts' });
  const address = accounts[0];

  // Setup ethers provider and signer
  const web3Provider = new ethers.providers.Web3Provider(window.ethereum);
  const web3Signer = web3Provider.getSigner();
  const network = await web3Provider.getNetwork();

  setProvider(web3Provider);
  setSigner(web3Signer);
  setChainId(network.chainId);
  setWallet(address);

  // Get ETH balance
  const balance = await web3Provider.getBalance(address);
  setEthBalance(ethers.utils.formatEther(balance));

  // Register with backend
  await axios.post(`${API}/investors/register`, { wallet_address: address });

  // Check network
  if (network.chainId !== 11155111) {
    toast.warning("Please switch to Sepolia testnet");
  }
}

const switchToSepolia = async () => {
  await window.ethereum.request({
    method: 'wallet_switchEthereumChain',
    params: [{ chainId: '0xaa36a7' }] // Sepolia chain ID
  });
};

// Deposit ETH to smart contract
const depositToContract = async (amountEth) => {
  const tx = await signer.sendTransaction({
    to: contractAddress,
    value: ethers.utils.parseEther(amountEth.toString()),
    data: "0xd0e30db0" // deposit() function selector
  });
  await tx.wait();
};

return (
  <WalletContext.Provider value={{
    wallet, provider, signer, chainId, ethBalance,
    connectWallet, switchToSepolia, depositToContract
  }}>
    {children}
  </WalletContext.Provider>
);

```

5. Smart Contract (AlphaAIManager.sol)

The AlphaAIManager smart contract handles on-chain fund management including investor deposits, withdrawals, and strategy allocations. Deployed on Ethereum Sepolia testnet.

5.1 Complete Solidity Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract AlphaAIManager {
    address public owner;

    struct Investor {
        uint256 balance; // simulated funds (in wei or token units)
        bool active;
    }

    struct Strategy {
        string name;
        uint256 allocated; // amount allocated to this strategy
        bool active;
    }

    mapping(address => Investor) public investors;
    mapping(uint256 => Strategy) public strategies;
    uint256 public strategyCount;

    event InvestorDeposited(address indexed investor, uint256 amount);
    event InvestorWithdrawn(address indexed investor, uint256 amount);
    event StrategyAdded(uint256 indexed strategyId, string name);
    event StrategyAllocated(uint256 indexed strategyId, uint256 amount);
    event StrategyDeallocated(uint256 indexed strategyId, uint256 amount);

    modifier onlyOwner() {
        require(msg.sender == owner, "Not authorized");
        _;
    }

    constructor() {
        owner = msg.sender;
    }

    /** Investor Functions */
    function deposit() external payable {
        require(msg.value > 0, "Deposit > 0 required");
        Investor storage inv = investors[msg.sender];
        inv.balance += msg.value;
        inv.active = true;
        emit InvestorDeposited(msg.sender, msg.value);
    }

    function withdraw(uint256 amount) external {
        Investor storage inv = investors[msg.sender];
        require(inv.balance >= amount, "Insufficient balance");
        inv.balance -= amount;
        if(inv.balance == 0) inv.active = false;
        payable(msg.sender).transfer(amount);
        emit InvestorWithdrawn(msg.sender, amount);
    }

    /** Strategy Functions */
    function addStrategy(string calldata name) external onlyOwner {
        strategies[strategyCount] = Strategy(name, 0, true);
        emit StrategyAdded(strategyCount, name);
        strategyCount++;
    }

    function allocateToStrategy(uint256 strategyId, uint256 amount) external onlyOwner {
        Strategy storage strat = strategies[strategyId];
        require(strat.active, "Strategy not active");
        strat.allocated += amount;
        emit StrategyAllocated(strategyId, amount);
    }

    function deallocateFromStrategy(uint256 strategyId, uint256 amount) external onlyOwner {
        Strategy storage strat = strategies[strategyId];
        require(strat.allocated >= amount, "Not enough allocated");
        strat.allocated -= amount;
        emit StrategyDeallocated(strategyId, amount);
    }

    /** Utility Functions */
    function getInvestorBalance(address investor) external view returns (uint256) {
        return investors[investor].balance;
    }
}
```

```

function getStrategy(uint256 strategyId) external view returns (string memory, uint256, bool) {
    Strategy storage strat = strategies[strategyId];
    return (strat.name, strat.allocated, strat.active);
}

// Emergency stop: owner can withdraw all funds (for testnet only)
function emergencyWithdraw() external onlyOwner {
    payable(owner).transfer(address(this).balance);
}
}

```

5.2 Contract Functions & Events

Function	Type	Description
deposit()	payable	Deposit ETH into the fund. Emits InvestorDeposited event
withdraw(uint256 amount)	external	Withdraw ETH. Requires sufficient balance
addStrategy(string name)	onlyOwner	Add a new trading strategy
allocateToStrategy(uint256 id, uint256 amt)	onlyOwner	Allocate funds to a strategy
deallocateFromStrategy(uint256 id, uint256 amt)	onlyOwner	Deallocate from strategy
getInvestorBalance(address)	view	Get investor balance
getStrategy(uint256 id)	view	Get strategy details
emergencyWithdraw()	onlyOwner	Emergency fund withdrawal

5.3 Contract Events

Event	Parameters	Triggered When
InvestorDeposited	address investor, uint256 amount	User deposits ETH
InvestorWithdrawn	address investor, uint256 amount	User withdraws ETH
StrategyAdded	uint256 strategyId, string name	New strategy added
StrategyAllocated	uint256 strategyId, uint256 amount	Funds allocated
StrategyDeallocated	uint256 strategyId, uint256 amount	Funds deallocated

5.4 Deployment Guide

```

Step 1: Get Sepolia ETH from a faucet (https://sepoliafaucet.com)
Step 2: Open Remix IDE (https://remix.ethereum.org)
Step 3: Create new file "AlphaAImanager.sol" and paste contract code
Step 4: Select Solidity compiler 0.8.20 and compile
Step 5: In Deploy tab, select "Injected Provider - MetaMask"
Step 6: Ensure MetaMask is connected to Sepolia network
Step 7: Click "Deploy" and confirm transaction in MetaMask
Step 8: Copy deployed contract address
Step 9: Register contract via POST /api/contract/register with:
{contract_address, deployer_address, tx_hash}

```

6. Trading Strategies & Signal Logic

6.1 Strategy Types

Type	Description	Signal Logic
Momentum	Trend following strategy	signal = btc_return * 1.5 if positive, * 0.8 if negative
Arbitrage (Mean Reversion)	Trades against large moves	signal = -btc_return if btc_return > 2%
Volatility Breakout	Trades on volatility spikes	signal = btc_return * 2 * direction
Adaptive	Mix of strategies	signal = btc_return * 0.7 + eth_return * 0.3
Funding Rate	Exploits funding differentials	Trades when funding rate > 0.01%
Yield	DeFi yield optimization	Optimizes across protocols for best APY

6.2 Trading Rules (from research simulation)

```
# Agent configuration for trading
agents = [
    {"name": "Arbitrage Agent", "allocation": 0.25, "strategy": "mean_reversion", "lookback": 20},
    {"name": "Momentum Agent", "allocation": 0.25, "strategy": "trend_following", "lookback": 10},
    {"name": "Funding Rate Agent", "allocation": 0.25, "strategy": "volatility_breakout", "lookback": 3},
    {"name": "AI Research Lab", "allocation": 0.25, "strategy": "adaptive", "lookback": 7}
]

# Strategy-based trading logic
for agent in agents:
    agent_capital = capital * agent["allocation"]

    if agent["strategy"] == "mean_reversion":
        # Trade against large moves (contrarian)
        signal = -btc_return if abs(btc_return) > 0.02 else btc_return * 0.5

    elif agent["strategy"] == "trend_following":
        # Follow the trend with enhanced position sizing
        signal = btc_return * 1.5 if btc_return > 0 else btc_return * 0.8

    elif agent["strategy"] == "volatility_breakout":
        # Trade on volatility spikes
        signal = abs(btc_return) * (1 if btc_return > 0 else -1) * 2

    else: # adaptive
        # Mix of strategies based on market conditions
        signal = btc_return * 0.7 + eth_return * 0.3

    # Calculate P&L based on signal and position
    position_size = agent_capital * 0.15 # 15% position
    trade_pnl = position_size * signal * random.uniform(0.8, 1.2)
```

6.3 Backtesting Logic

```
# Backtest a strategy on historical data
async def backtest_strategy(strategy_id: str, request: BacktestRequest):
    strategy = await db.strategies.find_one({"id": strategy_id})

    # Simulate backtest results
    total_return = round(random.uniform(-5, 35), 2)
    sharpe = round(random.uniform(0.5, 2.5), 2)
    max_dd = round(random.uniform(2, 10), 2)
    win_rate = round(random.uniform(45, 75), 1)
    total_trades = random.randint(50, 500)

    backtest_results = {
        "period": f"{request.start_date} to {request.end_date}",
        "initial_capital": request.initial_capital,
        "final_capital": round(request.initial_capital * (1 + total_return/100), 2),
        "total_return": total_return,
        "sharpe_ratio": sharpe,
        "max_drawdown": max_dd,
        "win_rate": win_rate,
        "total_trades": total_trades,
    }
```

```
# Update strategy with results
await db.strategies.update_one(
    {"id": strategy_id},
    {"$set": {
        "status": "backtested",
        "backtest_results": backtest_results,
        "sharpe_ratio": sharpe,
        "total_return": total_return
    }}
)

return {"success": True, "results": backtest_results}
```

7. AI Integration & Prompts

7.1 AI Market Analysis

The platform uses OpenAI GPT-5.2 via the Emergent LLM Key for market analysis. The AI provides sentiment analysis, support/resistance levels, and trading recommendations.

```
from emergentintegrations.llm.chat import LlmChat, UserMessage

EMERGENT_LLM_KEY = os.environ.get('EMERGENT_LLM_KEY')

@api_router.post("/ai/analyze")
async def ai_market_analysis(request: AIAnalysisRequest):
    # Get current market data
    market_data = await get_market_data(request.symbol.lower())
    price = market_data.get("market_data", {}).get("current_price", {}).get("usd", 0)
    change_24h = market_data.get("market_data", {}).get("price_change_percentage_24h", 0)

    try:
        # Initialize AI chat with Emergent integration
        chat = LlmChat(
            api_key=EMERGENT_LLM_KEY,
            session_id=f"analysis-{request.symbol}-{uuid.uuid4().hex[:8]}",
            system_message="You are an expert crypto trading analyst."
        ).with_model("openai", "gpt-5.2")

        # Create prompt for analysis
        user_message = UserMessage(
            text=f"Analyze {request.symbol.upper()} @ ${price:,.2f}, 24h: {change_24h:.2f}%. "
            f"Give: sentiment, levels, recommendation, risk (1-10)."
        )

        # Get AI response
        response = await chat.send_message(user_message)

        return {
            "symbol": request.symbol.upper(),
            "price": price,
            "change_24h": change_24h,
            "analysis": response,
            "timestamp": datetime.now(timezone.utc).isoformat()
        }
    except Exception as e:
        # Fallback if AI fails
        return {
            "symbol": request.symbol.upper(),
            "analysis": f"Market {'bullish' if change_24h > 0 else 'bearish'}. 24h: {change_24h:.2f}%"
        }
```

7.2 System Prompt

System Message: "You are an expert crypto trading analyst."

User Prompt Template:

"Analyze [SYMBOL] @ \${[PRICE]}, 24h: [CHANGE]%. Give: sentiment, levels, recommendation, risk (1-10)."

8. Database Schema (MongoDB)

The platform uses MongoDB with the Motor async driver. All collections store JSON documents with auto-generated string IDs (not ObjectIds) to avoid serialization issues.

Collection	Document Structure	Purpose
investors	{id, wallet_address, balance, shares, paper_balance, paper_pnl}	Investor accounts
trading_agents	{id, name, type, status, strategy, performance_7d, capital_allocation}	AI agents
trades	{id, agent_id, symbol, side, amount, price, pnl, slippage, is_simulation}	Trade history
strategies	{id, name, type, parameters, status, sharpe_ratio, total_return, is_active}	Trading strategies
risk_config	{max_drawdown, max_position_size, max_daily_loss, auto_shutdown_threshold}	Risk settings
risk_alerts	{id, type, severity, message, resolved, auto_action_taken}	Risk alerts
simulation_config	{is_running, mode, initial_capital, time_acceleration}	Simulation state
simulation_logs	{id, log_type, agent_name, message, details, timestamp}	Simulation logs
agent_interactions	{from_agent, to_agent, interaction_type, payload}	Agent-to-agent logs
marketplace_agents	{id, name, strategy, developer_address, performance_30d}	Marketplace agents
event_agents	{id, name, type, events_to_monitor, is_active}	Event-driven agents
contract_events	{event_name, block_number, tx_hash, args, processed}	Contract events
contract_info	{contract_address, network, deployed, deployer_address}	Contract deployment
research_simulations	{id, config, metrics, equity_curve, trade_count}	Research results
walk_forward_tests	{id, aggregate_metrics, window_results}	Walk-forward results
investor_reports	{report_id, executive_summary, performance_summary}	Generated reports
historical_data	{timestamp, symbol, open, high, low, close, volume}	Price history

9. Configuration Files

9.1 Backend Environment (.env)

```
# MongoDB connection
MONGO_URL=mongodb://localhost:27017
DB_NAME=alphaai_fund

# Emergent LLM Key for AI integration
EMERGENT_LLM_KEY=your_emergent_key_here

# CORS settings
CORS_ORIGINS=*
```

9.2 Frontend Environment (.env)

```
# Backend API URL
REACT_APP_BACKEND_URL=https://your-app.preview.emergentagent.com
```

9.3 Backend Dependencies (requirements.txt)

```
fastapi>=0.109.0
uvicorn>=0.27.0
motor>=3.3.0
python-dotenv>=1.0.0
httpx>=0.26.0
pydantic>=2.5.0
emergentintegrations>=1.0.0
reportlab>=4.0.0
web3>=6.0.0
pandas>=2.0.0
```

9.4 Frontend Dependencies (package.json key deps)

```
{
  "dependencies": {
    "react": "^18.2.0",
    "react-router-dom": "^6.x",
    "axios": "^1.x",
    "ethers": "^5.7.0",
    "recharts": "^2.x",
    "framer-motion": "^10.x",
    "sonner": "^1.x",
    "@radix-ui/react-*": "Shadcn/UI components",
    "lucide-react": "^0.x",
    "tailwindcss": "^3.x"
  }
}
```

10. End-to-End System Workflow

10.1 User Onboarding Flow

1. User visits landing page → Sees fund stats, features, live prices
2. User clicks "Connect Wallet" → MetaMask popup appears
3. User approves connection → Frontend calls POST /api/investors/register
4. Backend creates investor record → Returns investor data
5. User redirected to Dashboard → Can view portfolio, enable paper trading

10.2 Simulation Flow

1. Admin navigates to Simulation page → Configures settings
2. Clicks "Start Simulation" → POST /api/simulation/start
3. Backend initializes SimulationEngine → Creates default strategies
4. Runs capital allocation → Distributes funds to strategies
5. Each "Run Cycle" → Executes trades for each strategy
6. After each trade → Risk engine checks rules
7. If risk violated → Auto-stop or reduce positions
8. Results logged to database → Displayed on dashboard

10.3 Strategy Lab Flow

1. User opens Strategy Lab → Views existing strategies
2. Selects type + clicks "Generate Strategy" → POST /api/lab/strategies/generate
3. AI generates strategy → Status = "generated"
4. User clicks "Backtest" → POST /api/lab/strategies/{id}/backtest
5. Backtest runs → Returns Sharpe, return, drawdown
6. If good metrics → User clicks "Sandbox"
7. Sandbox testing runs → Paper trades strategy
8. If Sharpe > 1.0 and positive returns → User clicks "Deploy"
9. Strategy goes live → Receives capital allocation

10.4 Research Engine Flow

1. User opens Research page → Configures simulation
2. Sets target months, speed (500x), capital
3. Clicks "Run Simulation" → POST /api/research/run-simulation
4. Backend loads historical data from CSV files
5. Runs 4 agents through historical prices
6. Calculates daily P&L, equity curve, metrics
7. Returns comprehensive results with charts
8. User can run Walk-Forward Test → Validates robustness
9. Generate Investor Report → Creates PDF/JSON

10.5 Smart Contract Flow

1. Admin deploys AlphaAIManager.sol to Sepolia
2. Registers contract address → POST /api/contract/register
3. User connects MetaMask on Sepolia network
4. User clicks "Deposit" → Prepares transaction via backend
5. User signs transaction in MetaMask → ETH sent to contract
6. Contract emits InvestorDeposited event
7. Event agents detect event → Process automatically
8. Investor Monitor Agent → Updates balances dashboard
9. Strategy Allocator Agent → Auto-allocates 25% to top strategy

10.6 Event-Driven Agent Flow

1. Contract event occurs (deposit, withdrawal, allocation)
2. Backend receives event → Stores in contract_events collection
3. process_event() function called → Gets active agents
4. Each agent matching event type → Executes action
5. Watcher agents → Update balances, log activity
6. Execution agents → Auto-allocate funds to strategies
7. Analytics agents → Refresh dashboards
8. Agent stats updated → events_processed_count incremented



© 2026 Martin Maughan. All rights reserved. AlphaAI Platform.
This document is for external review and system rebuilding purposes.